

Reviewer Pack — Minimal Rank of Hodge Diamond over Function Fields vs BP_Rea...

Entry 7806a7903860 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 21:41:17 UTC

Minimal Rank of Hodge Diamond over Function Fields vs BP_ReadTwice Circuit Depth

Entry ID: 7806a7903860 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-24 21:41:17 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Hodge Theory) **Field B** (complexity object): Complexity Theory: Branching Program Complexity

Statement:

[‘For every read-twice BP P with polynomially many variables, the Hodge diamond of its defining polynomial has a minimal rank that is linear in the depth of P .’, ‘If there exists an instance of a BP P such that the minimal rank of its defining polynomial’s Hodge diamond is significantly lower than linear in its depth.’, ‘The Hodge diamond of the defining polynomial for a BP P has a minimal rank of $\Theta(d(P))$ where $d(P)$ is the depth of P .’]

Rationale (proposer’s reasoning):

[‘Hodge theory, which studies algebraic varieties and their cohomology groups, has been previously underutilized in complexity theory. The Hodge diamond provides a rich structure that could encode combinatorial information about computational problems like BP.’, ‘The conjecture suggests a possible link between the geometric properties of polynomials and the combinatorial complexity of BPs. If true, it would provide new insights into the hardness of computation.’]

Taxonomy category: BP_READTWICE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): cb969d928b0d9d37

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, after analyzing 1200 BP P instances with depth $d(P) \leq 40$ across 30 random seeds, the correlation coefficient between the minimal rank of the Hodge diamond and the depth is ≥ 0.8 AND the mean absolute error in the linear regression model for minimal rank vs. depth is ≤ 5 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Hodge diamond" AND "function fields" AND "BP_ReadTwice circuit depth" - "minimal rank" AND "Hodge diamond" AND "complexity theory" - "algebraic geometry" AND "Hodge Theory" AND "branching program complexity"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random

def generate_bp(depth):
    if depth == 1:
        return [random.choice([0, 1])]
    sub_depth = random.randint(2, depth - 1)
    left = generate_bp(sub_depth)
    right = generate_bp(depth - sub_depth - 1)
    return [left + right]

def compute_hodge_diamond(bp):
    n = len(bp)
    hodge_diamond = [[0] * (n + i) for i in range(n)]
    hodge_diamond[0][0] = bp[0]
    for i in range(1, n):
        hodge_diamond[i][0] = hodge_diamond[i-1][i-1]
        hodge_diamond[i][i] = bp[i]
        for j in range(1, i):
            hodge_diamond[i][j] = hodge_diamond[i-1][j-1] + hodge_diamond[i-1][j]
    return hodge_diamond

def min_rank(hodge_diamond):
    n = len(hodge_diamond)
    rank = 0
    for i in range(n):
        row_sum = sum(hodge_diamond[i])
        if row_sum != 0:
            rank += 1
    return rank
```

```

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_tests = 30
    total_rank = 0
    total_depth = 0
    instances_tested = 0

    for _ in range(n_tests):
        depth = random.randint(5, 40)
        bp = generate_bp(depth)
        hodge_diamond = compute_hodge_diamond(bp)
        rank = min_rank(hodge_diamond)

        total_rank += rank
        total_depth += depth
        instances_tested += 1

    mean_rank = total_rank / instances_tested
    mean_depth = total_depth / instances_tested
    correlation_coefficient = (instances_tested * sum(rank * depth for rank, depth in zip([mean_rank, mean_depth]))) / (instances_tested * instances_tested)
    mean_absolute_error = sum(abs(rank - depth) for rank, depth in zip([mean_rank, mean_depth]) * instances_tested) / instances_tested

    conjecture_holds = correlation_coefficient >= 0.8 and mean_absolute_error <= 5
    counterexample = "" if conjecture_holds else "correlation_coefficient={:.2f}, mean_absolute_error={:.2f}"

    return {
        "metric_name": "Correlation Coefficient",
        "metric_value": correlation_coefficient,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(seed) for seed in sys.argv[1:]] if sys.argv[1:] else [2**i - 1 for i in range(5, 10)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print("TRIAL:", {"seed": seed, **result})
        results.append(result)

    mean_correlation_coefficient = sum(r["metric_value"] for r in results) / len(results)
    mean_absolute_error = sum(abs(r["metric_value"] - 1) for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print("RESULT: SUPPORTED mean={:.2f} std={:.2f} support_fraction={:.2f}".format(mean_correlation_coefficient, mean_absolute_error, support_fraction))
    elif support_fraction >= 0.8:
        print("RESULT: SUPPORTED mean={:.2f} std={:.2f} support_fraction={:.2f}".format(mean_correlation_coefficient, mean_absolute_error, support_fraction))
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print("RESULT: FALSIFIED counterexample=\"correlation_coefficient={:.2f}, mean_absolute_error={:.2f}\"".format(mean_correlation_coefficient, mean_absolute_error))

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9a1b99e0.py", line 85, in <module>
    result = run_trial(seed)
             ~~~~~
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9a1b99e0.py", line 55, in run_trial
    bp = generate_bp(depth)
         ~~~~~
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9a1b99e0.py", line 22, in generate_bp
    left = generate_bp(sub_depth)
           ~~~~~
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9a1b99e0.py", line 22, in generate_bp
    left = generate_bp(sub_depth)
           ~~~~~
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9a1b99e0.py", line 22, in generate_bp
    left = generate_bp(sub_depth)
           ~~~~~
[Previous line repeated 4 more times]
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9a1b99e0.py", line 21, in generate_bp
    sub_depth = random.randint(2, depth - 1)
                ~~~~~
File "/usr/lib/python3.12/random.py", line 336, in randint
    return self.randrange(a, b+1)
           ~~~~~
File "/usr/lib/python3.12/random.py", line 319, in randrange
    raise ValueError(f"empty range in randrange({start}, {stop})")
ValueError: empty range in randrange(2, 2)
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete its intended analysis of BP P instances to verify the conjecture. | next: Re-run the test with proper error handling and ensure that it completes the analysis without crashing.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11408
2	preregistration	ollama_remote	glm4:latest	0	0	5703
3	novelty	ollama_remote	glm4:latest	0	0	4967
4	novelty	ollama_remote	glm4:latest	0	0	5484
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19382
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8697
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11129
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12411
9	judge	ollama_remote	glm4:latest	0	0	10899

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 90081 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/7806a7903860.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/7806a7903860.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/7806a7903860.tar.gz` (if generated)